



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

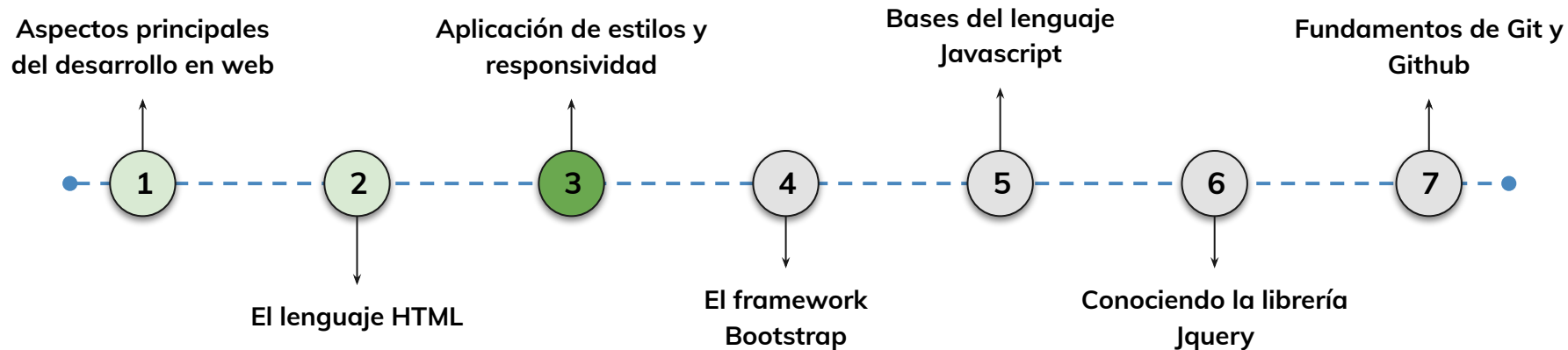


› Aplicación de estilos y responsividad - Parte I

AE 3.1: Aplicar hojas de estilo css básicas distinguiendo elementos de responsividad para personalizar la presentación de un documento html acorde a un requerimiento entregado.

Hoja de ruta

¿Cuáles skills conforman el programa?



Learning Path

¿Cuáles temas trabajaremos hoy?

AE3.1

Diseño Web con CSS

Aprenderás a aplicar estilos con CSS, utilizando selectores y reglas, además de estructurar mejor los elementos con Flexbox.

Fundamentos de CSS

Selectores y reglas CSS (por etiqueta, clase, ID)

Diferencia entre estilos en línea, embebidos y archivos externos

Diseño con Flexbox

Alineación y distribución de elementos con Flexbox

Creación de un menú de navegación con Flexbox



Objetivos de aprendizaje

¿Qué aprenderás?



- Poder crear una hoja de estilos CSS para trabajar el diseño web
- Aprender como utilizar selectores y reglas de CSS para aplicar estilos
- Comprender la diferencia entre: Estilos en línea, embebidos, archivos externos
- Buenas prácticas al construir una hoja de estilos y manejo de assets e imágenes.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Crear una estructura básica de formulario
- Comprender el uso de div
- Comprender la estructura de HTML
- Entender el uso y beneficios de span



En la escala del pato, cómo te sientes hoy?



Check-In

¿Cómo comenzamos hoy?

Qué es CSS, fundamentos y utilidad

< > Qué es CSS, fundamentos y utilidad

Se utiliza para definir cómo se deben mostrar los elementos HTML en términos de diseño, formato, colores y disposición. CSS se basa en selectores, propiedades y valores para aplicar estilos a los elementos HTML, permitiendo una separación clara entre contenido y diseño.

Esto resulta en sitios web visualmente atractivos y bien estructurados.



Qué es CSS, fundamentos y utilidad

Puntos claves:

Reglas CSS: Las reglas CSS consisten en un selector que elige elementos HTML y un conjunto de propiedades y valores que definen su estilo.

Selectores: Los selectores en CSS, como clases o identificadores, determinan a qué elementos se aplicarán las reglas de estilo.

Puntos claves:

Herencia y Cascada: CSS sigue un modelo de cascada, lo que significa que los estilos pueden heredarse de elementos padres a elementos hijos y pueden ser sobrescritos según la especificidad.

Enlace Externo: En la práctica, se vinculan archivos CSS externos a documentos HTML utilizando la etiqueta `<link>` en la sección `<head>` del HTML.

< > Estilos en línea, embebidos, archivos externos

Estilos embebidos (Embedded Styles):

Definición: Los estilos embebidos se definen dentro del elemento `<style>` en la sección `<head>` de un documento HTML. Los estilos se aplican a los elementos que coincidan con los selectores definidos en ese documento.

```
<!DOCTYPE html>
<html>
  <head>
    <style>
      p {
        color: blue;
        font-size: 18px;
      }
    </style>
  </head>
  <body>
    <p>Este es un párrafo con estilos embebidos.</p>
  </body>
</html>
```



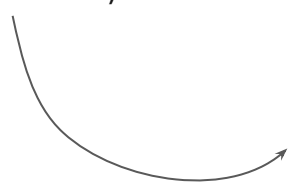
< > Estilos en línea, embebidos, archivos externos

Archivos externos (External Stylesheets):

Definición: Los archivos externos CSS se crean como archivos separados con extensión .css y se vinculan a un documento HTML utilizando la etiqueta <link> en la sección <head>.

Los estilos se aplican a todos los elementos que coincidan con los selectores definidos en el archivo CSS.

Ejemplo: (archivo estilos.css)



```
p {  
  color: green;  
  font-size: 20px;  
}
```



< > Referencias y selectores, por clase, por id

Selección por etiqueta (selector de tipo):

Definición: es una forma de seleccionar todos los elementos HTML de un tipo específico y aplicar estilos a esos elementos. Este tipo de selector utiliza el nombre de la etiqueta HTML como criterio de selección.

El selector por etiqueta es útil cuando deseas aplicar un estilo general a un tipo específico de elemento en todo tu sitio web.

Sin embargo, es importante tener en cuenta que afectará a todos los elementos de esa etiqueta en la página, por lo que debes usarlo con precaución y considerar si es necesario o si es mejor utilizar selectores más específicos, como selectores por clase o por ID, para elementos particulares.



< > Referencias y selectores, por clase, por id

(Hoja css)

```
p{  
  color: blue;  
  font-weight: bold;  
}
```

(Hoja html)

```
<p>Este es un párrafo destacado.</p>  
<p>Este es otro párrafo destacado.</p>
```



< > Referencias y selectores, por clase, por id

(Hoja css)

```
.destacado{  
  color: blue;  
  font-weight: bold;  
}
```

(Hoja html)

```
<p class="destacado">Este es un párrafo destacado.</p>  
<p class="destacado">Este es otro párrafo destacado.</p>
```



< > Referencias y selectores, por clase, por id

(Hoja css)

```
#miElemento {  
  color: blue;  
  font-size: 16px;  
}
```

(Hoja html)

```
<h1 id="#miElemento">Este es el título principal</h1>
```





LIVE CODING

Ejemplo en vivo

Migración de estilos en línea a estilos externos:

Contexto: Estamos trabajando en la optimización de un sitio web y nos dimos cuenta de que tiene demasiados estilos en línea (in-line) en su HTML. Esto dificulta el mantenimiento y la organización del código. Nuestra tarea es migrar estos estilos en línea a un archivo CSS externo para mejorar la estructura y el rendimiento del sitio.

Objetivo: Durante este live coding, migraremos estilos en línea a un archivo CSS externo y comprenderemos por qué es importante hacerlo y cómo esta práctica mejora la mantenibilidad y la organización del código.

Tiempo: 20 minutos

LIVE CODING

Ejemplo en vivo

```
<div style="color: red; font-size: 20px;">Este es un texto rojo con un tamaño de fuente de 20px.</div>
<p style="background-color: yellow; text-align: center;">Este es un párrafo con fondo amarillo y texto centrado.</p>
<a href="#" style="text-decoration: underline; background-color: lightblue;">Este es un enlace</a>
<h1 style="background-color: gray; color: white;">Este es un encabezado con fondo gris y texto blanco.</h1>
<p style="color: green; background-color: yellow;">Este es un párrafo con texto verde sobre fondo amarillo.</p>
<span style="color: red; background-color: pink;">Este es un texto rojo sobre fondo rosa.</span>
<div style="color: orange; background-color: gray;">Este es un texto naranja sobre fondo gris.</div>
<p style="color: white; background-color: black;">Este es un párrafo con texto blanco sobre fondo negro.</p>
```

LIVE CODING

Ejemplo en vivo

Solución paso a paso:

1. Identificación de estilos en línea

- Abre el archivo HTML del sitio web y transcribir los estilos en línea de la página anterior
- **Ejemplo:** `<div style="color: red;">.`

2. Creación de un archivo CSS externo

- Crea un nuevo archivo CSS en la misma carpeta que tu archivo HTML.
- Puedes llamarlo, por ejemplo, `styles.css`.

Solución Paso a Paso:

3. Extracción de Estilos en Línea

- Copia los estilos en línea que encontraste en el paso 1 y pégalos en el archivo CSS externo (styles.css).
- Los estilos en línea se deben traducir a reglas CSS válidas.
- Por ejemplo, si tienes `<div style="color: red;">`, en el archivo CSS sería `div { color: red; }`.

4. Vinculación del Archivo CSS

- Abre el archivo HTML y elimina los estilos en línea que copiaste al archivo CSS.
- Vincula el archivo CSS externo a tu HTML utilizando la etiqueta `<link>` en el `<head>` del documento HTML. Debe verse así: `<link rel="stylesheet" href="styles.css">`.



Momento:

Time-out!



5 -10 min.



< > El modelo de Cajas

Existen diferentes formas de especificar los márgenes en CSS:

Margen individual: Se puede especificar márgenes individualmente para cada lado del elemento utilizando las propiedades `margin-top`, `margin-right`, `margin-bottom` y `margin-left`.

```
.elemento {  
  margin-top: 10px;  
  margin-right: 20px;  
  margin-bottom: 10px;  
  margin-left: 20px;  
}
```

En este caso, se establecen márgenes de 10 píxeles en la parte superior e inferior, y márgenes de 20 píxeles en los lados derecho e izquierdo del elemento.



< > El modelo de Cajas

Existen diferentes formas de especificar los márgenes en CSS:

```
.elemento {  
  margin: 10px 20px 10px 20px; /* arriba, derecha,  
  abajo e izquierda */  
}  
  
.elemento {  
  margin: 10px 20px; /* arriba y abajo, derecha e  
  izquierda */  
}  
  
.elemento {  
  margin: 10px; /* en todos los puntos por igual */  
}
```

Margen abreviado: También se puede utilizar la propiedad margin para especificar los márgenes de manera abreviada.

Puedes proporcionar uno, dos, tres o cuatro valores separados por espacios. Los valores se aplicarán en el orden de arriba, derecha, abajo e izquierda



< > El modelo de Cajas

Existen diferentes formas de especificar los márgenes en CSS:

Margen automático: Utilizar “margin: auto” permite centrar horizontalmente un elemento dentro de su contenedor. Esto ajustará automáticamente los márgenes laterales para que el elemento esté centrado.

```
.elemento {  
  margin: auto;  
}
```

Los márgenes también pueden tener valores negativos, lo que permite que los elementos se superpongan entre sí o se ajusten a un diseño específico. Es importante tener en cuenta que los márgenes se acumulan entre elementos adyacentes. Esto significa que si hay varios elementos uno al lado del otro, los márgenes se sumarán, lo que puede afectar el espacio total entre ellos.



< > El modelo de Cajas

Existen diferentes formas de especificar el relleno (Padding) en CSS:

Padding individual: Se puede especificar el padding individualmente para cada lado del elemento utilizando las propiedades padding-top, padding-right, padding-bottom y padding-left.

En este caso, se establece un padding de 10 píxeles en la parte superior e inferior, y un padding de 20 píxeles en los lados derecho e izquierdo del elemento.

```
.elemento {  
  padding-top: 10px;  
  padding-right: 20px;  
  padding-bottom: 10px;  
  padding-left: 20px;  
}
```



< > Fuentes

Font-family: Esta propiedad se utiliza para especificar el tipo de fuente que se utilizará para el texto. Se puede utilizar nombres de fuentes específicos o una lista de fuentes genéricas, como "Arial", "Helvetica", "Times New Roman", etc. También es posible definir múltiples fuentes en caso de que una no esté disponible en el dispositivo del usuario.

```
.selector {  
  font-family: Arial, sans-serif;  
}
```



< > Fuentes

Font-size: Permite establecer el tamaño de la fuente. Se puede utilizar unidades de píxeles, puntos, porcentaje, em o rem para especificar el tamaño. Al elegir un tamaño de fuente, es importante considerar la legibilidad en diferentes dispositivos y tamaños de pantalla.

Text-align: Esta propiedad se utiliza para alinear el texto dentro de su contenedor. Se puede alinear el texto al centro, a la izquierda, a la derecha o justificarlo.

```
.selector {  
  font-size: 16px;  
  text-align: center;  
}
```



< > Fuentes

font-weight: Esta propiedad controla el grosor o la negrita del texto. Se puede utilizar valores como "normal", "bold", "bolder", etc. para establecer diferentes niveles de peso en la fuente.

line-height: Se utiliza para establecer la altura de línea, es decir, el espacio vertical entre líneas de texto. Un valor adecuado para line-height puede mejorar la legibilidad y el flujo visual del texto.

```
.selector {  
  font-weight: bold;  
  line-height: 1.5;  
}
```



< > Flexbox

El modelo de caja flexible se basa en dos conceptos clave:

- el contenedor flex (flex container)
- los elementos flexibles (flex items).

Para utilizar Flexbox, debes seguir estos pasos:

Aplica la propiedad `display: flex` ; al contenedor que deseas convertir en un contenedor flex.

Esto define un nuevo contexto de formato flexible para sus elementos secundarios.

```
.selector {  
  display: flex;  
}
```



< > Box-shadow

Configurar la dirección y el flujo:

Utiliza la propiedad **flex-direction** para establecer la dirección de los elementos secundarios dentro del contenedor flex. Puede ser **row (horizontal)**, **column (vertical)**, **row-reverse** o **column-reverse**.

Utiliza la propiedad **flex-wrap** para permitir que los elementos secundarios se envuelvan en múltiples líneas si no caben en una sola línea.

```
.selector {  
  display: flex;  
  flex-direction: row;  
}
```

```
.selector {  
  display: flex;  
  flex-wrap: wrap;  
}
```



< > Box-shadow

Controlar la distribución y el espacio:

Utiliza la propiedad **justify-content** para alinear los elementos secundarios a lo largo del eje principal del contenedor flex. Puede ser **flex-start**, **flex-end**, **center**, **space-between**, **space-around** o **space-evenly**.

Utiliza la propiedad **align-items** para alinear los elementos secundarios a lo largo del eje transversal del contenedor flex. Puede ser **flex-start**, **flex-end**, **center**, **stretch** o **baseline**.

Utiliza la propiedad **align-content** para controlar el espacio entre las líneas de elementos secundarios si hay varias líneas dentro del contenedor flex. Puede ser: **flex-start**, **flex-end**, **center**, **space-between**, **space-around**, y **stretch**.

```
.selector {  
  display: flex;  
  justify-content: center;  
}
```

```
.selector {  
  display: flex;  
  align-items: flex-start;  
}
```

```
.selector {  
  display: flex;  
  align-content: space-between;  
}
```



LIVE CODING

Ejemplo en vivo

Estilización de la Tipografía:

A partir de lo aprendido, vamos a usar las propiedades de CSS relacionadas con la tipografía, para crear una página web simple utilizando HTML y CSS donde aplicaremos estilos de fuente a elementos de texto específicos.

El objetivo es practicar el uso de propiedades de CSS relacionadas con la tipografía.

Tiempo: 15 minutos

LIVE CODING

Ejemplo en vivo

Pasos:

1. Crear un archivo HTML llamado index.html y un archivo CSS llamado styles.css.
2. En el archivo HTML, crear la estructura básica de una página web con un encabezado (<header>), un párrafo (<p>), una lista desordenada () con al menos tres elementos de lista (), y un pie de página (<footer>).
3. Dentro del <header>, agregar un título (<h1>) y un subtítulo (<h2>).
4. En el archivo CSS (styles.css), enlazar las fuentes de Google Fonts para usar en la página web. Por ejemplo, utilizar la fuente "Roboto" para el texto principal y la fuente "Open Sans" para el encabezado.

LIVE CODING

Ejemplo en vivo

Pasos:

5. Personalizar el color de fuente, el espaciado entre líneas (line-height), y otros estilos de fuente según preferencias.
6. Agregar cualquier otro estilo de fuente adicional, como el uso de colores, tamaños de fuente y márgenes.
7. Asegurarnos de enlazar el archivo CSS en el archivo HTML utilizando la etiqueta `<link>` en la sección `<head>`.
8. Visualizar nuestra página web en un navegador y ajustar los estilos de fuente según sea necesario para lograr el aspecto deseado.
9. Agregar las sesiones que sean necesarias.



Ejercicio N° 1

Incorporando CSS

Incorporando CSS

Contexto: 🙌

Eres un desarrollador web contratado para mejorar el aspecto y la estructura de un sitio web existente. El cliente desea que el sitio sea más atractivo visualmente y siga las mejores prácticas de desarrollo web.

Consigna: 📝

Mejorar los estilos y la estructura del sitio web utilizando HTML y CSS. Utilizarás selectores CSS para aplicar estilos basados en ID y clases en el HTML existente.

Tiempo 🕒: 10 minutos

Incorporando CSS

Paso a paso: 

1. Preparación

- Abre el proyecto web anterior existente y revisa su estructura actual.
- Crea un archivo CSS separado si aún no lo tienes y vincularlo al archivo HTML.
- Asegúrate de que todas las imágenes y recursos necesarios estén disponibles y funcionando correctamente.

2. Mejora de la Estructura HTML

- Revisa el HTML y asegúrate de que esté bien estructurado con etiquetas semánticas como `<header>`, `<nav>`, `<section>`, `<article>`, `<footer>`, etc.
- Agrega IDs o clases a los elementos HTML que deseas estilizar de manera específica, a partir de lo entendido en clase, dale nombres representativos a cada semántica.



Ejercicio N° 2

Aplicando Flexbox

Aplicando Flexbox

Consigna: 🛠️

Crea una página web con un menú de navegación horizontal utilizando Flexbox.

El objetivo es aplicar las propiedades de Flexbox para alinear los elementos del menú horizontalmente.

Tiempo 🕒: 20 minutos

Aplicando Flexbox

Paso a paso:

1. Crea un archivo HTML llamado index.html y un archivo CSS llamado styles.css.
2. En el archivo HTML, crea la estructura básica de la página web. Debe incluir un encabezado (`<header>`) y un contenedor (`<nav>`) para el menú de navegación.
3. Dentro del contenedor de navegación (`<nav>`), agrega una lista desordenada (`<ul>`) con al menos cinco elementos de lista (`<li>`). Cada elemento de lista debe contener un enlace (`<a>`) que represente un elemento del menú (por ejemplo, Inicio, Acerca de, Servicios, Portafolio, Contacto).
4. En el archivo CSS (styles.css), aplica estilos básicos al cuerpo (`<body>`) de la página, como una fuente de Google Fonts y una paleta de colores.

Aplicando Flexbox

Paso a paso: ⚙️

5. Utiliza Flexbox para crear un menú de navegación horizontal. Aplica las propiedades `display: flex` y `justify-content` para alinear los elementos del menú horizontalmente.
6. Agrega espaciado y margen entre los elementos del menú para mejorar la apariencia.
7. Personaliza el diseño del menú según tus preferencias, como el tamaño de fuente, el color de fuente y los efectos de hover.
8. Visualiza tu página web en un navegador de escritorio para asegurarte de que el menú sea atractivo y funcional en una pantalla grande.
9. Publica tu proyecto en un servidor web o plataforma de alojamiento para que otros puedan verlo en línea si lo deseas.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <link rel="stylesheet" href="styles.css">
  <title>Menú de Navegación</title>
</head>
<body>
  <header>
    <h1>Logo</h1>
    <nav>
      <ul>
        <li><a href="#">Inicio</a></li>
        <li><a href="#">Acerca de</a></li>
        <li><a href="#">Servicios</a></li>
        <li><a href="#">Portafolio</a></li>
        <li><a href="#">Contacto</a></li>
      </ul>
    </nav>
  </header>
</body>
</html>
```

Solución

Parte 1

```
body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
}

header {
    background-color: #333;
    color: #fff;
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 10px 20px;
}

nav ul {
    list-style-type: none;
    display: flex;
}

nav ul li {
    margin-right: 20px;
}
```

```
nav ul li:last-child {
    margin-right: 0;
}

nav a {
    text-decoration: none;
    color: #fff;
    font-weight: bold;
    transition: color 0.3s ease;
}

nav a:hover {
    color: #f00;
}
```

Solución

Parte 2

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ **Comprender el uso de CSS para incorporar y trabajar el diseño web**
- ✓ **Comprender y utilizar estilos en línea, embebidos, archivos externos**
- ✓ **Incorporar flexbox a nuestro CSS**
- ✓ **Aplicar buenas prácticas para construir una hoja de estilos**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compártela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

